References

Capabilities & Limitations

- NON `const` References

```
int   i = 2;
int& ri = i;  // reference to i
```

ri and **i** refer to the same object / memory location:

```
cout << i  <<'\n';   // 2
cout << ri <<'\n';   // 2

i = 5;
cout << i  <<'\n';   // 5
cout << ri <<'\n';   // 5

ri = 88;
cout << i  <<'\n';   // 88
cout << ri <<'\n';   // 88
```

- references cannot be "null", i.e., they must always refer to an object
- a reference must always refer to the same memory location
- reference type must agree with the type of the referenced object

```
int  i  = 2;
int  k  = 3;
int& ri = i;      // reference to i

ri = k;           // assigns value of k to i (target of ri)

int& r2;          // ✗ COMPILER ERROR: reference must be initialized

double& r3 = i;   // ✗ COMPILER ERROR: types must agree
```

- `const` References

= Read-Only Access To An Object

```
int i = 2;
int const& cri = i;  // const reference to i
```

- **cri** and **i** refer to the same object / memory location
- but const means that value of **i** cannot be changed through **cri**

```
cout << i   <<'\n';   // 2
cout << cri <<'\n';   // 2

i = 5;
cout << i   <<'\n';   // 5
cout << cri <<'\n';   // 5

cri = 88;  // ✗ COMPILER ERROR: const!
```

- auto References

reference type is deduced from right hand side of assignment

```
int i = 2;

double d = 2.023;

double x = i + d;

auto & ri = i;         // ri:  int &

auto const& crx = x;  // crx: double const&
```

Usage

- Range based in loops

```
std::vector<std::string> v;
v.resize(10);

// modify vector elements:
for (std::string & s : v) { cin >> s; }
// read-only access to vector elements:
for (std::string const& s : v) { cout << s; }

// modify:
for (auto & s : v) { cin >> s; }
//read-only access:
for (auto const& s : v) { cout << s; }
```

- const Reference Parameters

Read-Only Access ⇒ **const&**

- avoids expensive copies
- clearly communicates read-only intent to users of function

Example: Function that computes median

❗ only needs to *read* values from vector!

✗ pass by **value** ⇒ copy	✓ pass by **const&** ⇒ no copy
`int median (vector<int>);`	`int median (vector<int> const&);`

`auto v = get_samples("huge.dat");`
`auto m = median(v);`
`// runtime & memory overhead!`
	`auto v = get_samples("huge.dat");`
	`auto m = median(v);`
	`// no copy ⇒ no overhead!`

Example: Mixed passing (by ref + by value)

`incl_first_last ({1,2,4},{6,7,8,9}) → {1,2,4,6,9}`

The implementation works on a local copy 'x' of the first vector and only reads from the second vector via const reference 'y':

```
auto incl_first_last (std::vector<int> x, std::vector<int> const& y) {
  if (y.empty() return x;
  // append to local copy 'x'
  x.push_back(y.front());
  x.push_back(y.back());
  return x;
}
```

- NON `const` References Parmeters

Example: Function that exchanges values of two variables

```
void swap (int& i, int& j) {
  int temp = i;  // copy i's value to temp
  i = j;         // copy j's value to i
  j = temp;      // copy temp's (i's original value) to j
}

int main () {
  int a = 5;
  int b = 3;
  swap(a,b);
  cout << a << '\n'   // 3
       << b << '\n';  // 5
}
```

Use **std::swap** to exchange values of objects (#include <utility>).

It can be used like the function above, but avoids expensive temporary copies for "move-enabled" objects like std::vector (it's implementation will be explained in chapter Move Semantics).

As useful as non-const references might be in some situations, you should avoid such "output parameters" in *general* (see the next panels for more details).

Usage

- Range based in loops

```
std::vector<std::string> v;
v.resize(10);

// modify vector elements:
for (std::string & s : v) { cin >> s; }
// read-only access to vector elements:
for (std::string const& s : v) { cout << s; }

// modify:
for (auto & s : v) { cin >> s; }
//read-only access:
for (auto const& s : v) { cout << s; }
```

- const Reference Parameters

Read-Only Access ⇒ **const&**

- avoids expensive copies
- clearly communicates read-only intent to users of function

Example: Function that computes median

❗ only needs to *read* values from vector!

✗ pass by **value** ⇒ copy	✓ pass by **const&** ⇒ no copy
`int median (vector<int>);`	`int median (vector<int> const&);`

`auto v = get_samples("huge.dat");`
`auto m = median(v);`
`// runtime & memory overhead!`
	`auto v = get_samples("huge.dat");`
	`auto m = median(v);`
	`// no copy ⇒ no overhead!`

Example: Mixed passing (by ref + by value)

`incl_first_last ({1,2,4},{6,7,8,9}) → {1,2,4,6,9}`

The implementation works on a local copy 'x' of the first vector and only reads from the second vector via const reference 'y':

```
auto incl_first_last (std::vector<int> x, std::vector<int> const& y) {
  if (y.empty() return x;
  // append to local copy 'x'
  x.push_back(y.front());
  x.push_back(y.back());
  return x;
}
```

- NON `const` References Parmeters

Example: Function that exchanges values of two variables

```
void swap (int& i, int& j) {
  int temp = i;  // copy i's value to temp
  i = j;         // copy j's value to i
  j = temp;      // copy temp's (i's original value) to j
}

int main () {
  int a = 5;
  int b = 3;
  swap(a,b);
  cout << a << '\n'   // 3
       << b << '\n';  // 5
}
```

Use **std::swap** to exchange values of objects (#include <utility>).

It can be used like the function above, but avoids expensive temporary copies for "move-enabled" objects like std::vector (it's implementation will be explained in chapter Move Semantics).

As useful as non-const references might be in some situations, you should avoid such "output parameters" in *general* (see the next panels for more details).

- Function Parmeters

```
void read_from (int);  // fundamental types
void read_from (std::vector<int> const&);
void copy_sink (std::vector<int>);
void write_to  (std::vector<int> &);
```

Read from cheaply copyable object (all fundamental types) ⇒ **pass by value**

```
double sqrt (double x) { … }
```

Read from object with larger (> 64bit) memory footprint ⇒ pass by **const&**

```
void print (std::vector<std::string> const& v) {
  for (auto const& s : v) { cout << s << ' '; }
}
```

Copy needed inside function anyway ⇒ **pass by value**

Pass by value instead of copying explictly inside the function. The reasons for this will be explained in more advanced articles.

```
auto without_umlauts (std::string s) {
  s.replace('ö', "oe");  // modify local copy
  …
  return s;  // return by value!
}
```

Write to function-external object ⇒ pass by non-const**&**

👉 As useful as they might be in some situations, you should avoid such “output parameters” *in general*, see here why.

```
void swap (int& x, int& y) { … }
```

- Avoid Output Parmeters

Functions with non-const ref parameters like

```
void foo (int, std::vector<int>&, double);
```

can create confusion/ambiguity at the call site:

```
foo(i, v, j);
```

- Which of the arguments (`i`, `v`, `j`) is changed and which remains unchanged?
- How and when is the referenced object changed and is it changed at all?
- Does the reference parameter only act as output (function only writes to it) or also as input (function also reads from it)?

⇒ in general **hard to debug and to reason about!**

👎 Example: An interface that creates nothing but confusion

```
void bad_minimum (int x, int& y) {
  if (x < y) y = x;
}
int a = 2;
int b = 3;
bad_minimum(a,b);
// Which variable holds the smaller value again?
```

Binding Rules

- R-Value VS L-Value

Lvalues = expressions of which we **can** get memory address

- refer to objects that persist in memory
- everything that has a name (variables, function parameters, ...)

Rvalues = expressions of which we **can't** get memory address

- literals (`123`, `"string literal"`, ...)
- temporary results of operations
- temporary objects returned from functions

```
int a = 1;      // a and b are both lvalues
int b = 2;      // 1 and 2 are both rvalues
a = b;
b = a;

a = a * b;      // (a * b) is an rvalue
int c = a * b;  // OK

a * b = 3;      // ✗ COMPILER ERROR: cannot assign to rvalue

std::vector<int> read_samples(int n) { … }
auto v = read_samples(1000);
```

- Reference Binding Rules

`&`	only binds to **L**values
`const&`	binds to **const L**values *and* **R**values

```
bool is_palindrome (std::string const& s) { … }

std::string s = "uhu";
cout << is_palindrome(s) <<", "
     << is_palindrome("otto") <<'\n';  // OK, const&
```

```
void swap (int& i, int& j) { … }

int i = 0;
swap(i, 5);  // ✗ COMPILER ERROR: can't bind ref. to literal
```

Pitfalls

- Never Return a Reference to a Function-*local* Object

```
int& increase (int x, int delta) {
    x += delta;
    return x;
}  // ↯ local x destroyed

int main () {
  int i = 2;
  int j = increase(i,4);  // ↯ accesses invalid reference!
}
```

Only valid if referenced object *outlives* the function!

```
int& increase (int& x, int delta) {
    x += delta;
    return x;  // x references non-local int
}  // OK, reference still valid

int main () {
  int i = 2;
  int j = increase(i,4);  // OK, i and j are 6 now
}
```

- Careful With Referencing `vector` elements

⚠ **References to elements of a `std::vector` might be invalidated after any operation that changes the number of elements in the vector!**

```
vector<int> v {0,1,2,3};
int& i = v[2];
v.resize(20);
i = 5; // ↯ UNDEFINED BEHAVIOR: original memory might be gone!
```

Dangling Reference = Reference that refers to a memory location that is no longer valid.

The internal memory buffer where `std::vector` stores its elements can be exchanged for a new one during some vector operations, so any reference into the old buffer might be dangling.

- Avoid Lifetime Extensions

References can extend the lifetime of temporaries (**r**values)

```
auto const& r = vector<int>{1,2,3,4};
```

⇒ vector exists as long as reference `r` exists

What about an object returned from a function?

```
std::vector<std::string> foo () { … }
```

✓ take it by value (recommended):

```
vector<string> v1 = foo();
auto v2 = foo();
```

😐 ignore it ⇒ gets destroyed right away

```
foo();
```

😐 get const reference to it ⇒ lifetime of temporary is extended

… for as long as the reference lives

```
vector<string> const& v3 = foo();
auto const& v4 = foo();
```

✗ **don't** take a reference to its members!

⚠ No lifetime extension for members of returned objects (here: the vector's content)!

```
string const& s = foo()[0];  // dangling reference!
cout << s;                   // ↯ UNDEFINED BEHAVIOR
```

👉 Don't use lifetime extension through references!

- easy to create confusion
- easy to write bugs
- no real benefit

Just take returned objects by value. This does not involve expensive copies for most functions and types in modern C++, especially in C++17 and above.

References

https://hackingcpp.com/cpp/lang/references.html